

Aula 1 — Arquitetura do PIC18

Quatro perguntas que a bancada vai fazer, e o que dentro do chip as responde

Microcontroladores — DENE/UFMT

OBJETIVOS

- Calcular o tempo de execução a partir da frequência do oscilador e identificar o erro de fator quatro que atravessa todo o semestre.
- Reconhecer os bits de configuração que produzem sintomas parecidos com defeito de programa.
- Explicar por que uma escrita em memória aciona um pino, localizando os registradores de periférico no mapa de dados.
- Ler o consumo de Flash e de RAM no mapa gerado pelo ligador e antecipar qual das duas se esgota primeiro.

1 Toda arquitetura decide coisas por você

Um microcontrolador não é uma folha em branco. Antes de vocês escreverem a primeira linha de código, alguém já decidiu quanto o chip conta por segundo, quanta memória existe e como ela é dividida, e em que estado os pinos acordam quando a placa é ligada. Esse conjunto de decisões é o que se chama de **arquitetura**, e ela tem uma propriedade incômoda: não se negocia. O programa é que tem de caber nela.

Isso não é particularidade do PIC18. ESP32, STM32, RISC-V — todos têm arquitetura, todos decidem exatamente as mesmas coisas, e todos decidem de forma diferente. Trocar de plataforma não elimina o problema; troca os números. É por isso que vale aprender a **fazer as perguntas**, e não decorar as respostas de um dispositivo específico.

Na prática, a arquitetura chega até o trabalho de vocês por três portas:

O que a arquitetura decide	O que isso custa a vocês
Quanto tempo custa cada instrução	Todo cálculo de temporização do semestre parte daí: piscar um LED, gerar um PWM, fixar uma taxa de comunicação. Errar esse número invalida todos os outros de uma vez
Como a memória é organizada, e o que está ligado atrás de cada endereço	Explica por que uma atribuição comum aciona um pino, e determina qual recurso vai acabar primeiro quando o projeto crescer
O que já está configurado antes da primeira instrução executar	Produz falhas que se parecem com defeito de programa, e que nenhuma releitura do código revela

NOTA

Guardem a terceira linha. Ela é a que mais custa tempo de laboratório, e quase nunca por dificuldade conceitual — por descuido de configuração. O sintoma aponta para o lugar errado.

1.1 As quatro perguntas de hoje

Na aula de laboratório, vocês vão ligar quatro atuadores usando apenas saída digital: LEDs, buzzer, lâmpada e cooler. O que se pretende nesta aula não é descrever o dispositivo de cima a baixo, mas responder antecipadamente às quatro perguntas que aquela bancada vai levantar — cada uma delas uma instância de uma das três portas acima.

Pergunta	Porta	Onde é respondida
----------	-------	-------------------

Por que o LED pisca em ritmo diferente do calculado?	Tempo	Seção 2
Por que um botão na porta B parece não funcionar?	Configuração pré-via	Seção 3
Por que escrever LATD = 0xFF acende oito LEDs?	Memória	Seção 4
Por que o programa vai esbarrar na RAM antes da Flash?	Memória	Seção 5

NOTA

A seção 6 reúne o restante da arquitetura — pilha, contador de programa, paralelismo, banqueamento — como contexto. Ela explica **por que as coisas são assim**, não **o que fazer na bancada**.

1.2 O dispositivo desta disciplina

O que está dentro do encapsulamento, para consulta:

Recurso	PIC18F4550
Núcleo	8 bits, arquitetura Harvard, multiplicador por hardware
Memória de programa	32 KB de Flash — 16 384 instruções de uma palavra
Memória de dados	2 048 bytes de RAM estática
Memória não volátil de dados	256 bytes de EEPROM
Frequência máxima do dispositivo	48 MHz, equivalentes a 12 MIPS
Frequência do núcleo nesta bancada	16 MHz, equivalentes a 4 MIPS
Portas de entrada e saída	35 pinos no encapsulamento de 40 vias
Conversor analógico-digital	10 bits, 13 canais
Comparadores	Dois, com referência programável
Temporizadores	Quatro: um de 8 bits e três de 16
Comunicação	Serial assíncrona, síncrona mestre-escravo e controlador USB

As duas linhas de frequência não são um erro de digitação. A diferença entre elas é o assunto da próxima seção.

2 Por que o LED pisca no ritmo errado

2.1 O ciclo de instrução

CONCEITO

Cada ciclo de instrução consome **quatro** ciclos do oscilador:

$$T_{cy} = \frac{4}{f_{osc}}$$

Com o núcleo a 16 MHz, como nesta bancada:

$$T_{cy} = \frac{4}{16 \cdot 10^6} = 250 \text{ ns}$$

ou seja, 4 milhões de instruções por segundo. Uma rotina de 120 instruções sem desvios executa em cerca de 30 μs.

ATENÇÃO

Confundir frequência do oscilador com frequência de instrução é um erro frequente. Ele reaparece nos encontros de temporizadores e de modulação, porque **todos** os divisores partem de T_{cy} , nunca de f_{osc} . Um erro de fator quatro num período é quase sempre este.

2.2 A árvore de clock, e por que ela é complicada

O PIC18F4550 tem uma árvore de clock desproporcional para um dispositivo de 8 bits, e a razão é o USB — periférico que este projeto não usa, mas cuja existência todos pagam na configuração.

O controlador USB exige exatamente 48 MHz, gerados por um multiplicador interno que precisa receber, na entrada, exatamente 4 MHz. Daí a cadeia:

1. o cristal externo entra no dispositivo;
2. um primeiro divisor, `PLLDIV`, reduz essa frequência aos 4 MHz exigidos;
3. o multiplicador gera a frequência alta interna;
4. um segundo divisor, `CPUDIV`, define a partir dela a frequência entregue ao processador.

O último passo é o que separa as duas linhas da tabela da seção 1: o multiplicador entrega 48 MHz, o USB os consome, e `CPUDIV` divide essa frequência antes de entregá-la ao núcleo. Nesta bancada, a divisão resulta em **16 MHz**.

DIVERGÊNCIA

Quem decide isso na XM118 não é o programa de vocês.

A placa vem com um **bootloader** gravado, e é ele que fixa as palavras de configuração — inclusive `PLLDIV`, `CPUDIV` e `FOSC`. Um bloco `#pragma config` escrito na aplicação compila sem erro, é gravado sem reclamação e **não tem efeito nenhum**: os bits já estão definidos e o bootloader é o dono deles.

A consequência prática para o laboratório é uma linha só:

```
#define _XTAL_FREQ 16000000UL
```

Se esse valor estiver em `48000000UL`, o compilador calculará os laços de `__delay_ms` para uma CPU três vezes mais rápida do que a real, e todos os atrasos sairão **três vezes mais longos**. É por isso que a oficina pede cronômetro: vinte piscadas medidas são a evidência mais barata de que o clock está correto.

TAREFA

Na aula prática, com `_XTAL_FREQ` correto, o período medido de um `__delay_ms(500)` dobrado bate com 1 s? Troque deliberadamente para `48000000UL`, meça de novo e confirme o fator três. Leve o cronômetro.

3 Por que o botão na porta B não funciona

Um botão ligado à porta B não é detectado. O mesmo código, com o botão movido para a porta D, funciona. O programa está correto — e está mesmo.

A causa é `PBADEN`, um bit de configuração que decide se os pinos da porta B nascem analógicos ou digitais. O padrão **não** é digital. Enquanto o pino estiver configurado como entrada analógica, a leitura digital devolve algo que não corresponde ao botão.

CONCEITO

As palavras de configuração são gravadas junto com o programa e definem o comportamento do dispositivo **antes da primeira instrução executar**. Nenhum erro nelas é detectado pelo compilador, e nenhum se manifesta como erro de compilação: todos se manifestam como comportamento estranho em execução.

Bit	Por que ele aparece nesta lista
XINST	Habilita o conjunto estendido de instruções, não suportado pelo compilador usado. Ligado por engano, o programa executa lixo. Deve permanecer desabilitado
LVP	Programação em baixa tensão. Habilitada, mantém um pino da porta B reservado, que deixa de funcionar como entrada e saída comum — e um ruído nesse pino pode colocar o dispositivo em modo de programação
WDT	Cão de guarda. Habilitado sem que o programa o alimente, provoca reinicializações periódicas que se manifestam como “o programa reinicia sozinho”
PBADEN	Define se pinos da porta B iniciam como analógicos ou digitais. O padrão não é digital: um botão ligado a esses pinos parece não funcionar até que isso seja corrigido
MCLRE	Define se o pino de reinicialização externa é reset ou entrada digital. Alterá-lo sem necessidade pode impedir a gravação
FOSC , PLLDIV , CPUDIV	Definem a frequência efetiva. Errados, corrompem toda a temporização e a comunicação serial

```
/* Bloco de configuracao tipico de uma placa SEM bootloader.
   Na XM118 estes valores sao fixados pelo bootloader e as
   diretivas abaixo, se escritas na aplicacao, sao ignoradas. */
#pragma config PLLDIV   = 5           /* conferir o cristal da placa */
#pragma config CPUDIV   = OSC1_PLL2
#pragma config FOSC     = HSPLL_HS
#pragma config WDT      = OFF         /* ligado somente na Aula 10 */
#pragma config LVP      = OFF         /* libera o pino da porta B */
#pragma config XINST    = OFF         /* obrigatorio com XC8 */
#pragma config PBADEN   = OFF         /* porta B digital no reset */
#pragma config MCLRE    = ON
```

ATENÇÃO

Vale registrar o método por trás desta seção, mais do que a lista. Nenhum desses bits é conceitualmente difícil; todos produzem sintomas que **parecem** defeitos de programa. Quando um código correto se comporta de maneira inexplicável, verifique a configuração antes de reescrever o código.

4 Por que LATD = 0xFF acende oito LEDs

Na oficina, uma atribuição a uma variável de aparência comum acendeu uma barra de LEDs. Isso não é uma abstração da biblioteca: LATD é literalmente um endereço de memória, e há hardware ligado atrás dele.

CONCEITO

No vocabulário da Microchip, **toda** posição de memória de dados é um registrador — um *file register*. Não existe um banco de registradores separado da RAM, como em ARM ou RISC-V. MOVWF 0x20 e MOVWF LATD são a mesma instrução, com endereços diferentes.

O que muda é o que está ligado atrás do endereço. Escrever num GPR apenas guarda um número; escrever num SFR **muda o comportamento do chip**.

Faixa	Nome	O que é
0x000 – 0x7FF	GPR	RAM de uso geral. Suas variáveis. Nada de hardware atrás
0x800 – 0xF5F	—	Não implementado. Lê zero; escrita é descartada
0xF60 – 0xFFF	SFR	Registradores de função especial: TRIS , LAT , PORT , ADCON , T0CON , TXREG , STATUS , WREG , BSR

Os 2 048 bytes de GPR são organizados em bancos de 256 bytes, porque as instruções não têm bits suficientes para endereçar 2 048 posições diretamente. O endereço completo se forma concatenando o banco selecionado em **BSR** com o deslocamento contido na instrução.

NOTA

Metade da memória de dados deste dispositivo é fisicamente compartilhada com o controlador USB. Com o módulo desabilitado — o caso deste projeto — a região fica disponível como memória de uso geral. É peculiaridade do dispositivo, não da família.

4.1 Três registradores por porta

Registra- dor	Papel
TRISx	Direção do pino: 1 é entrada, 0 é saída
LATx	O que você escreve — o valor mantido no latch de saída
PORTx	O que você lê — o estado elétrico do pino

ATENÇÃO

No *reset*, todo pino nasce como entrada e **LAT** é indefinido. Habilitar a saída em **TRIS** antes de escrever um estado seguro em **LAT** faz o pino comandar o atuador com lixo, por um breve intervalo.

Com um LED isso é invisível. Com um cooler ou uma resistência de aquecimento, não é. Daí a regra: **LAT antes de TRIS**.

Na aula de laboratório, vocês vão inverter a ordem deliberadamente e pressionar o *reset* várias vezes, prestando atenção no cooler. Anotem agora o que esperam ouvir.

4.2 Para quem vem de programação em computador

NOTA

Escrever `int x;` num programa de computador é pedir espaço a um sistema operacional que decide, em tempo de execução, onde a variável vai morar. O endereço muda a cada execução, é virtual, e o programador nunca precisa saber qual é.

Aqui não há sistema operacional, não há memória virtual e não há tempo de execução para decidir nada: o compilador escolhe um endereço físico fixo — digamos 0x022 — e é lá que `x` vive, sempre, em todas as execuções. Essa é a diferença que organiza todas as outras.

Con- ceito	No computador	No PIC18
Variável local	Nasce e morre num quadro de pilha; endereço só existe em execução	Endereço fixo dado pelo compilador, reaproveitado entre funções que não coexistem

Pilha	Uma só, em RAM, guarda retorno, argumentos e locais	Só endereços de retorno, em silício dedicado, 31 níveis; não consome RAM
Ponteiro inválido	Falha de segmentação; o programa morre	Nada acontece — ou um pino comuta, ou um periférico muda de modo
Alocação dinâmica	Rotineira	Praticamente inexistente; 2 KB não comportam heap
Memória disponível	Gigabytes	2 048 bytes, verificados pelo ligador em tempo de compilação

ATENÇÃO

Duas consequências merecem ser guardadas desde já.

A primeira é a **não reentrância**: se as variáveis locais têm endereço fixo, a mesma função chamada do laço principal e de um tratador de interrupção usará as mesmas posições de memória nas duas invocações — e uma sobrescreverá a outra. Isso é retomado na Aula 9.

A segunda é a **ausência de rede de proteção**. Num computador, o erro de memória se manifesta como um programa que morre; aqui, como um programa que continua rodando e faz outra coisa. O sintoma não aponta para a causa.

4.3 O que o compilador coloca em cada lugar

Construção em C	Onde reside	Observação
Código das funções	Flash	Consome memória de programa
<code>const</code> e literais de texto	Flash	Acesso por leitura de tabela
Variáveis globais e estáticas	RAM	Existem durante toda a execução
Variáveis locais	RAM	Endereços fixos, atribuídos pelo compilador
Variáveis de tratador	RAM, com <code>volatile</code>	Vide Aula 9
Dados persistentes	EEPROM	Escrita explícita; vide Aula 10

5 Por que a RAM acaba antes da Flash

O dispositivo tem 32 KB de Flash e 2 KB de RAM — uma proporção de dezesseis para um. O laboratório acrescenta uma camada por semana ao mesmo código, e a experiência de outros semestres é consistente: a memória de programa raramente é o limite deste projeto; os 2 KB de RAM são.

ATENÇÃO

Buffers de comunicação, vetores de média móvel e cadeias de texto consomem RAM rapidamente. Um vetor de 64 amostras de 16 bits, sozinho, já toma 128 bytes — mais de 6% do total.

A boa notícia é que o estouro é reportado pelo **ligador**: é erro de compilação, não falha em execução.

Que os dois números sejam independentes — que uma cadeia de texto constante grande não roube espaço das variáveis — decorre de os dois espaços serem separados. É este o sentido operativo de **arquitetura Harvard** nesta disciplina: memória de programa e memória de dados com espaços de endereçamento distintos e barreiros próprios, podendo ter larguras diferentes. Aqui, 16 bits para programa e 8 para dados.

CONCEITO

A separação cobra um preço. Como `const` mora na Flash e a Flash não é endereçável como dado, ler uma constante exige instruções específicas de leitura de tabela (`TBLRD`), não um simples acesso à memória. O compilador cuida disso, mas o custo em ciclos é real e aparece na listagem.

5.1 Do código-fonte ao chip

	Ferramenta	Produto
1	Pré-processador	Código com diretivas resolvidas
2	Compilador	Código de máquina relocável, por módulo
3	Ligador	Programa único, com endereços resolvidos
4	Gerador de imagem	Arquivo <code>.hex</code> , com o conteúdo da Flash
5	Gravador	Dispositivo programado

6 Contexto: o resto da arquitetura

Nada nesta seção é necessário para a bancada. Está aqui porque explica **por que** as coisas anteriores são como são, e porque o vocabulário reaparece no seminário comparativo, quando o PIC18 for colocado ao lado de ARM Cortex-M e RISC-V.

CONTEXTO

Contador de programa e vetores. O contador de programa tem 21 bits, capazes de endereçar 2 MB; o dispositivo implementa 32 KB. Três endereços têm significado fixo: `0x0000` é o vetor de reinicialização, `0x0008` o de interrupção de alta prioridade e `0x0018` o de baixa. Entre os dois últimos cabem apenas dezesseis bytes — oito instruções —, o que não comporta um tratador. O que o compilador coloca ali é um desvio para o tratador propriamente dito, alocado em outro ponto.

CONTEXTO

Pilha de hardware. Chamadas de função e interrupções empilham o endereço de retorno numa pilha dedicada de 31 níveis, separada da memória de dados: consumir pilha não consome RAM. Estourar 31 níveis é difícil em código bem estruturado, mas possível com recursão. Diferentemente da família anterior, o PIC18 permite detectar o estouro. Um reset inexplicável em código com recursão tem aí a primeira hipótese.

CONTEXTO

Paralelismo de busca e execução. Como programa e dados usam barramentos distintos, a próxima instrução pode ser buscada enquanto a atual é executada. O processador completa, assim, uma instrução por ciclo. A exceção são os desvios: eles descartam a instrução já buscada e custam dois ciclos — detalhe que importa apenas quando se contam ciclos para temporização precisa.

CONTEXTO

Banco de acesso. Trocar de banco a cada acesso custa ciclos. O PIC18 abre uma janela virtual de 256 bytes que reúne os 96 primeiros bytes de GPR aos SFR do topo do mapa; toda instrução tem um bit que seleciona essa janela. O resultado é que variáveis muito usadas e todos os registradores de periférico ficam acessíveis em uma única instrução, sem tocar em `BSR`. Combinado ao endereçamento indireto e ao multiplicador por hardware, é o que torna o PIC18 uma arquitetura **projetada** para receber código de compilador C, e não apenas tolerante a ele.

6.1 Transposição

Quem já viu a família PIC16 reconhecerá quase tudo, com quatro diferenças: o vetor único de interrupção deu lugar a dois, com prioridade; a pilha passou de 8 para 31 níveis e tornou-se observável; o banco de acesso

removeu a troca constante de banco; e surgiram o multiplicador por hardware e os modos de endereçamento indireto que viabilizam C eficiente.

Rumo às arquiteturas de 32 bits, dois pontos desta aula sobrevivem intactos e um desaparece. Sobrevivem a **árvore de clock** — ainda mais elaborada, com multiplicadores e divisores por periférico, e igualmente capaz de invalidar toda a temporização quando mal configurada — e o **orçamento de memória**, ainda lido no mapa do ligador. Desaparece o **banqueamento**: com endereçamento linear de 32 bits, bancos e janelas de acesso deixam de existir, e com eles toda uma classe de erros.

7 Antes de descer para o laboratório

Reserve os últimos dez minutos desta aula para preencher a tabela abaixo. Ela é a mesma da Oficina 1, e é a razão de as duas sessões estarem coladas: vocês preveem agora, com a teoria fresca, e verificam daqui a trinta minutos.

NOTA

Previsão errada não é problema. Previsão **feita depois** do experimento é: ela deixa de ser previsão e vira racionalização. Preençam antes de sair da sala, rubriquem, e levem a folha para a bancada.

Pergunta	Previsão e justificativa	Rubrica
P1. O que acontece com o cooler no instante do <i>reset</i> , antes de a primeira linha do programa executar?		
P2. Se TRIS for configurado antes de LAT , o comportamento muda? Como?		
P3. Piscando o cooler a 10 Hz, o que você espera ver e ouvir?		
P4. E a 1 kHz?		
P5. Piscar mais rápido faz o cooler girar mais devagar? Justifique.		

BANCADA

Leve para a bancada: esta folha preenchida, um cronômetro e o código de referência do Roteiro 0. Lembrete de segurança que será repetido lá: os pontos **LAMP**, **HEATER** e **COOLER** estão no trilho de 12 V, e nenhuma garra de terra de osciloscópio encosta neles.

CONCEITO

Voltando ao início. A aula abriu afirmando que a arquitetura decide coisas por vocês. Na aula de laboratório, essa afirmação vira observação: o cooler vai partir com um tranco se **TRIS** vier antes de **LAT**, o LED vai errar o ritmo por um fator três se **_XTAL_FREQ** estiver errado, e vocês vão descobrir que não existe meia velocidade com saída digital pura.

Esse último ponto não tem solução dentro do que foi visto hoje — e é intencional. Ele é a pergunta que abre o Roteiro 3.

8 Exercícios

TAREFA

Exercício 1.1. Um projeto usa cristal de 8 MHz e precisa que o processador opere a 24 MHz. Determine os divisores de entrada e de saída necessários, justificando a restrição que fixa a frequência na entrada do multiplicador. Calcule T_{cy} e o tempo de execução de uma rotina de 400 instruções sem desvios.

(b) Refaça o cálculo de T_{cy} para a configuração da bancada, com núcleo a 16 MHz, e diga em que fator os dois resultados diferem.

TAREFA

Exercício 1.2. Um estudante relata que o botão ligado a um pino da porta B não é detectado, embora o mesmo código funcione com o botão movido para a porta D. O programa está correto. Indique a causa mais provável, o bit envolvido e a correção.

TAREFA

Exercício 1.3. Considere um vetor de 64 amostras de 16 bits para média móvel, mais dois buffers de comunicação de 32 bytes cada e uma cadeia de texto constante de 200 caracteres. Calcule o consumo de RAM e o de Flash, indicando em qual memória cada item reside e que fração dos recursos disponíveis é consumida.

TAREFA

Exercício 1.4. Na oficina, o cooler foi acionado por um laço que alterna 500 μ s ligado e 500 μ s desligado usando `__delay_us`.

(a) Quantos ciclos de instrução, a 16 MHz, correspondem a cada meio período?

(b) Que fração do tempo de processador esse laço consome?

(c) Um termostato precisa, no mesmo intervalo, ler o sensor de temperatura e decidir a ação. Explique por que este laço torna isso impossível e o que precisaria mudar na arquitetura do programa.

TAREFA

Exercício 1.5. Um colega escreveu `_XTAL_FREQ` como `48000000UL` e relata que o LED pisca “devagar demais”. Sem acesso à bancada, determine o fator de erro esperado e descreva o experimento mínimo, com cronômetro, que confirma a hipótese.

NOTA

A discussão sobre em que sentido “a arquitetura Harvard é mais rápida que a de Von Neumann” é correta, e em que sentido é uma simplificação indevida quando se comparam processadores de gerações diferentes, fica reservada ao **seminário comparativo** — momento em que vocês terão ao lado do PIC18 os dados de um Cortex-M e de um RISC-V para sustentar a comparação.